НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Департамент программной инженерии

ОТЧЁТ

по индивидуальному заданию №2
по дисциплине
«Архитектура компьютера и операционные системы»

Вариант 29. Группа лифтов

Разработка многопоточной имитационной программы

Выполнил: Буров Иван Васильевич
Группа: БПИ252

Москва, 2026

1 Условие задачи

В многоэтажном здании работает несколько лифтов под управлением общего диспетчера. Пассажиры появляются на разных этажах через случайные интервалы, каждый задаёт этаж назначения и формирует вызов с этажом отправления и направлением. Диспетчер назначает вызов лифту по выбранной стратегии. Лифт не принимает людей сверх вместимости: если прибывшая кабина заполнена, оставшиеся продолжают ожидание, и вызов считается выполненным только после формирования нового вызова для них. Лифт обслуживает попутные вызовы; направление меняется после выполнения заявок текущего направления.

Входные параметры: число этажей, лифтов и вместимость кабин; начальное положение лифтов; количество пассажиров и их этажи; диапазон времени появления; скорость движения и время дверей, посадки и высадки; стратегия назначения вызовов.

Инварианты: вместимость не превышается; пассажир входит и выходит только при открытых дверях на этаже остановки; лифт не движется с открытыми дверями; пассажир находится либо на этаже, либо в одном лифте; выходит только на заданном этаже; один вызов не назначается нескольким лифтам.

Завершение: после доставки всех пассажиров либо по истечении заданного периода; для оставшихся выводится состояние их заявок.

2 Отображение сущностей в потоки

Каждую сущность, у которой есть своё поведение, я сделал отдельным потоком. Общего такта в программе нет: время идёт настоящее, и каждый поток сам выдерживает свои паузы вызовом `nanosleep`. При 25 пассажирах и трёх кабинах одновременно работает около 30 потоков.

Сущность	**Потоков**	**Что делает поток**
Пассажир	по одному	ждёт, пока его посадят, потом пока довезут
Лифт	по кабине	выбирает направление, едет, открывает двери, высаживает и сажает; когда работы нет — засыпает
Диспетчер	один	спит, пока не появится вызов, потом раздаёт все нераспределённые
Генератор	один	создаёт пассажиров через случайные промежутки
Главный	один	рисует экран, проверяет инварианты, следит за сигналами

Всё состояние модели лежит в одной структуре `Sim`: массивы кабин, пассажиров и вызовов, счётчики статистики. Её видят все потоки сразу.

Как живёт вызов. Пассажир появляется и либо нажимает кнопку, либо просто присоединяется к тем, кто уже ждёт: на этаж и направление бывает не больше одного активного вызова. Диспетчер отдаёт вызов одной кабине и назад уже не забирает — именно поэтому два лифта никогда не поедут на один вызов, то есть шестой инвариант выполняется сам собой. Приехавшая кабина забирает всех, кто едет в её сторону, даже

если их вызов был назначен другому лифту — это и есть попутные вызовы. После этого вызов гасится, а для тех, кто не поместился, сразу регистрируется новый.

3 Синхронизация потоков

Примитив	**Сколько**	**Зачем**
мьютекс `lock`	один	защищает всё состояние модели
семафор `new_call`	один	пассажир будит диспетчера
семафор `work`	по кабине	диспетчер будит простаивающий лифт
`boarded`, `arrived`	по два на пассажира	кабина сообщает ему, что он вошёл и что приехал

Один мьютекс на всё состояние я выбрал намеренно. Можно было завести отдельный замок на каждую кабину и на реестр вызовов, но выигрыш в скорости здесь не нужен, а риск появляется вполне реальный: два потока берут два замка в разном порядке и встают навсегда. С одним мьютексом такое просто невозможно.

Весь код написан по одной схеме: взять мьютекс и решить, что делать — отпустить — сделать долгое дело — снова взять и записать результат. Если бы кабина спала 400 мс переезда, не отпуская мьютекс, все остальные потоки стояли бы эти 400 мс. Лифты ездили бы по очереди, и многопоточная программа работала бы как обычная последовательная. Поэтому внутри критических секций у меня нет ни пауз, ни вывода на экран.

Простаивающий лифт не проверяет состояние в цикле, а засыпает на семафоре. Опрос раз в несколько миллисекунд при 25 пассажирских потоках дал бы тысячи захватов мьютекса в секунду впустую, и все они мешали бы кабинам работать. Важно ещё и то, что семафор считает сигналы: если лифт посмотрел, работы не нашёл, но заснуть не успел, а диспетчер как раз в этот момент назначил ему вызов — сигнал не пропадёт, счётчик семафора запомнит его.

Ждать семафор можно только с отпущенным мьютексом. Иначе получается тупик: пассажир заснёт, держа мьютекс, а кабине для посадки нужен тот же мьютекс — пассажир ждёт лифта, лифт ждёт мьютекс, и оба стоят вечно. А вот посылать сигнал под мьютексом не страшно, эта операция не блокирует.

Писал я на macOS, и там `sem_init` помечена устаревшей и возвращает `ENOSYS` — безымянные семафоры попросту не работают. Пришлось взять именованные: имя собираю из номера процесса и счётчика, а сразу после `sem_open` вызываю `sem_unlink`, чтобы при аварийном завершении в системе ничего не осталось. Такой код одинаково работает и на macOS, и на Linux.

4 Вторая программа: условные переменные

Всю синхронизацию я спрятал за четырьмя функциями (`sig_init`, `sig_wait`, `sig_post`, `sig_destroy`), поэтому вторая версия отличается ровно одним файлом. `make` собирает из одного и того же набора исходников два бинарника: `build/elevator-sem` на семафорах и `build/elevator-cond` на условных переменных.

Условная переменная устроена иначе. Вызов `pthread_cond_wait` атомарно отпускает мьютекс, засыпает и при пробуждении захватывает его обратно. Поэтому её, в отличие

от семафора, надо ждать *удерживая* мьютекс — правило прямо противоположное, и на нём легко запутаться.

Но главное отличие другое: условная переменная не считает сигналы. Сигнал, посланный когда никто не ждёт, пропадает бесследно. Я проверил это отдельной маленькой программой, где ожидание заведомо начинается позже сигнала:

```
условная переменная: через 1.5 с поток ВСЁ ЕЩЁ СПИТ
семафор: дождался
```

Для нашей модели это смертельно: лифт заснул бы навсегда при наличии работы. Поэтому счётчик я завёл руками — по сути собрал счётный семафор из условной переменной и мьютекса:

```
void sig_wait(Signal *s)
{
    pthread_mutex_lock(&s->lock);
    while (s->count == 0)
        pthread_cond_wait(&s->cond, &s->lock);
    s->count--;
    pthread_mutex_unlock(&s->lock);
}
```

Здесь `while`, а не `if`, потому что `pthread_cond_wait` имеет право вернуться сам по себе, без всякого сигнала. Проснулись — перепроверили условие.

Ради интереса я собрал вариант вообще без счётчика, ожидая, что он зависнет. Он не завис ни разу за сорок прогонов: окно, в котором сигнал теряется, шириной в наносекунды, а сигналов в программе много и они перекрывают друг друга. То есть такая версия не сломана явно — она работает на удачу и может изредка зависнуть на ровном месте. По-моему, это хуже честной ошибки.

Барьеры `pthread_barrier_*`, которые тоже предлагались в задании, на macOS отсутствуют целиком: `_POSIX_BARRIERS` равен -1, заголовков нет. Поэтому выбор и пал на условные переменные.

5 Проверка инвариантов

Все шесть инвариантов программа проверяет на ходу. Если что-то нарушено, в журнал идёт строка `INVARIANT BROKEN`, а программа завершается с кодом возврата 2.

Тактов здесь нет, поэтому проверки пришлось разделить. Два инварианта — про открытые двери и про этаж назначения — я проверяю прямо в момент действия, внутри функций посадки и высадки. По состоянию модели «потом» уже никак не понять, были ли двери открыты в тот миг, когда человек шагнул в кабину. Остальные четыре проверяются по снимку состояния: главный поток каждые `refresh_ms` берёт мьютекс — то есть видит согласованную картину, пока её никто не меняет, — и смотрит вместимость, движение с открытыми дверями, что пассажир находится ровно в одной кабине и что на этаж в одном направлении не появилось двух активных вызовов.

6 Пользовательский интерфейс

Экран перерисовывается каждые `refresh_ms`: сверху схема здания, снизу лента последних десяти событий.

```
2848 ms   delivered 2/14   nearest   threads 15

  fl   E1     E2     E3    hall
  10    .      .      .
   9    .    (v3)     .
   8    .      .      .
   7    .      .      .
   6    .      .      .
   5  [v1]     .      .
   4    .      .      .   2^
   3    .      .      .
   2    .      .    (^0)   2^
   1    .      .      .   1^

[  2593 ms] passenger 8 enters elevator 2
[  2610 ms] passenger 3 leaves elevator 3 on floor 2, delivered
[  2805 ms] dispatcher sends elevator 2 to the call on floor 1 (up)
```

`[v1]` — кабина с закрытыми дверями едет вниз, внутри один человек, выделяется жёлтым. `(^0)` — двери открыты, зелёным. Точка означает, что шахта на этом этаже пуста. Справа, в столбце `hall`, видно, сколько человек ждёт на этаже и в какую сторону.

Кадр я собираю в буфер, пока держу мьютекс, а на экран пишу уже после того, как его отпустил: запись в терминал — системный вызов, и делать его под мьютексом значило бы остановить всю программу. События потоки тоже не печатают сами: тридцать потоков просто затёрли бы картинку и перемешали строки. Вместо этого события копятся в кольцевом буфере, у которого свой мьютекс, отдельный от мьютекса модели. Флаг `--plain` отключает перерисовку и печатает события сплошной лентой, так удобнее для скриптов. Параллельно всё пишется в журнал через отдельный файловый дескриптор: параметры запуска, все события и итоговая статистика.

7 Варианты запуска и завершения

Параметры программа берёт из трёх мест, и каждое следующее перекрывает предыдущее: встроенные умолчания, конфигурационный файл, аргументы командной строки. И файл, и аргументы разбирает одна и та же функция, поэтому набор ключей у них всегда одинаковый. Все значения проверяются ещё до того, как запустится хоть один поток.

```
./build/elevator-sem                                  # умолчания
./build/elevator-sem --config data/rush-hour.conf     # сценарий из файла
./build/elevator-sem --config data/default.conf --floors 20 --strategy fcfs
./build/elevator-cond --plain --passengers 20 --seed 42
```

Завершиться программа может четырьмя способами, и все они заканчиваются одинаково — выводом статистики и списком незакрытых заявок: доставлены все пассажиры; истёк период `--max-seconds`; пришёл SIGINT или SIGTERM; пришёл SIGUSR1, по которому программа перестаёт впускать новых пассажиров и довозит тех, кто уже в здании.

Обработчик сигнала у меня выставляет одну-единственную переменную типа `volatile sig_atomic_` и больше не делает ничего. Иначе нельзя: сигнал прерывает какой-то произвольный поток, и тот вполне может в этот момент держать мьютекс, так что захватывать его или печатать из обработчика — верный способ получить тупик.

Самое неприятное место — само завершение. К этому моменту почти все потоки спят на семафорах, и если кого-то не разбудить, `pthread_join` будет ждать его вечно. Поэтому, сняв флаг работы, программа посылает сигнал каждому ожидающему: обоим семафорам каждого пассажира, семафору каждой кабины и семафору диспетчера. Проснувшись, поток первым делом смотрит на флаг и выходит. По замеру после Ctrl-C программа завершается примерно за 255 мс.

Коды возврата: 0 — всё нормально, 1 — ошибка в параметрах, 2 — нарушен инвариант.

8 Тестирование

В каталоге `data/` лежат пять сценариев, каждый одновременно и пример конфигурации, и тестовый набор: `small` (5 этажей, один лифт), `default` (12 этажей, 3 лифта), `rush-hour` (час пик, 16 этажей, 4 лифта), `crowded` (вместимость 1), `slow-doors` (медленные двери, стратегия `fcfs`).

Скрипт `./tests/run-tests.sh` делает 38 проверок и возвращает 0, только если прошли все: сборка обоих бинарников без предупреждений, отказ на семи вариантах неверного ввода, пять сценариев на каждой сборке, обе стратегии, краевые случаи, совпадение журнала с экраном, SIGINT и SIGUSR1 на обеих сборках, по 30 случайных конфигураций здания и прогон под ThreadSanitizer.

Многопоточную программу приходится проверять иначе, чем обычную. Один удачный запуск не доказывает ничего: ошибка синхронизации вылезает в одном прогоне из сотни. Поэтому я делаю три вещи. Во-первых, гоняю программу на множестве разных параметров. Во-вторых, использую жёсткий режим, где все задержки сокращены до нескольких миллисекунд — тогда потоки лезут в общее состояние гораздо чаще, и ошибки проявляются охотнее. Обе серьёзные ошибки, которые я нашёл, всплыли именно так. В-третьих, запускаю всё под ThreadSanitizer: он следит за каждым обращением к памяти и сообщает о переменных, с которыми два потока работают без синхронизации. Из трёх приёмов только он даёт не «не поймали», а «проверено».

Ещё одна мелочь: в macOS нет команды `timeout`, а основной вид отказа здесь — зависание. Поэтому в скрипт встроен собственный сторожевой таймер, который снимает прогон, если тот не уложился в срок.

9 Результаты работы программы

Все пять сценариев довезли всех пассажиров и ни разу не нарушили инварианты. Величины в миллисекундах.

Сценарий	Время	Дост.	Ожид.	Макс.	Поездка	Отказов
`small`	18158	8/8	5032	8708	2452	0
`default`	42666	25/25	2735	7277	3560	0
`rush-hour`	28460	60/60	3945	11982	3499	2
`crowded`	18676	20/20	5148	11090	1179	13
`slow-doors`	64811	25/25	10688	24243	8339	0

Цифры получились осмысленные. В `slow-doors` здание и поток пассажиров те же, что в базовом сценарии, но все длительности больше вдвое с лишним — и среднее ожидание выросло с 2,7 до 10,7 секунды. В `crowded` кабина возит по одному человеку: ожидание подскакивает до 5,1 секунды, 13 раз кто-то не поместился, зато сама поездка самая короткая из всех — здание низкое.

Одинаковый сценарий на двух сборках: `elevator-sem` отработал за 42827 мс при среднем ожидании 1805,6 мс, `elevator-cond` — за 42829 при 1809,4. Разница в пределах погрешности, чего и следовало ожидать: синхропримитивы меняют способ передачи сигналов, а не поведение модели.

Стратегии на одном и том же потоке пассажиров (`--seed 42`): `fcfs` — 40145 мс, среднее ожидание 2270,7, максимальное 6174; `nearest` — 39595, 2243,2 и 6779. Получается, `nearest` заканчивает раньше и в среднем люди ждут чуть меньше, но максимальное ожидание у неё хуже: отправляя к вызову ближайшую кабину, она сбивает кабины в одну часть здания, и кому-то дальнему приходится ждать дольше. По одному прогону делать выводы, конечно, нельзя.

Отдельно я проверил устойчивость: по 30 прогонов на каждую сборку со случайными конфигурациями (2–15 этажей, 1–5 лифтов, вместимость 1–5) в жёстком режиме. Все пассажиры доставлены, ни одного зависания, инварианты целы. ThreadSanitizer на обеих сборках не нашёл ни одной гонки данных.

10 Использование искусственного интеллекта

Задание я выполнял с ИИ-ассистентом Claude (Anthropic). Работали в диалоге: я задавал вопросы, уточнял, ставил требования, ассистент объяснял на примерах, помогал в написании решений. С его помощью спроектирована архитектура, сделаны тестовые сценарии со скриптом прогона и составлен этот отчёт. На мне были постановка задачи, написание кода, решение о том, что входит в работу и какого она объёма, и проверка результата каждого этапа.

Запросы, по которым появлялся код:

1. «Делаю второе дз. Проанализируй его, скажи, в чем отличия от первого. После этого детально пройдемся по необходимым темам и разберем на независимых примерах. Помогай мне писать код, если я прошу.».
2. «Подробнее про мьютексы и семафоры с примерами», «объясни по человечески, что делаем и почему» — разбор темы перед каждым этапом.
3. Несколько запросов вида «помоги написать код» и «как правильно реализовать ...», по одному на этап: каркас и обёртка над семафорами; потоки лифтов и диспетчера; потоки пассажиров и инварианты; экран, статистика и сигналы; вторая программа на условных переменных;.
4. «Напиши: тесты, README, комментарии, отчет, используй информацию из диалога»

11 Критическая оценка сгенерированного кода

Код, который получался, был далеко не сразу правильным. Ниже все ошибки, которые я нашёл при проверке, и то, как они исправлены.

1. **Одна кабина делала всю работу.** В одной из изначальных стратегий возникала проблема, что на вызов ехал один из лифтов, хотя расстояние от них до пассажира было одинаковым. Из-за чего один из лифтов постоянно простаивал. После добавления к стоимости числа уже назначенных кабине заявок, прогон стал занимать 1918 мс вместо 2811.
2. **Программа зависала намертво.** Из 25 конфигураций четыре повисли и были сняты по таймауту, все с одним лифтом вместимостью один человек. Кабина полна, назначить новый вызов некому, диспетчер засыпает. Лифт довозит пассажира и освобождается, но будить диспетчера нечем: новых вызовов не появлялось. Исправил: после высадки кабина сообщает диспетчеру, что место освободилось. Все четыре конфигурации стали проходить.
3. **Пассажир доставлялся дважды.** После перевода пассажиров на отдельные потоки в выводе появилось доставлено 13 из 12 и сработал инвариант. Генератор делал пассажира видимым для лифтов раньше, чем его поток выставлял состояние: лифт сажал его к себе, запустившийся поток затирал эту запись, и второй лифт сажал того же человека снова. Исправил: пассажир заполняется целиком под мьютексом и становится видимым только готовым.
4. **Статистика врала.** Прервал программу по Ctrl-C и увидел среднее ожидание 1159,5 мс при максимальном 1110. Сумма времён копилась для всех, кто сел в кабину, а делилась на число доехавших, хотя трое в тот момент ещё ехали. Завёл отдельный счётчик севших. При обычном завершении ошибка не проявлялась.
5. **Тест проходил вхолостую.** Три проверки упали из-за моей ошибки в аргументах. Разбирая их, заметил, что проверка совпадения журнала с экраном падает так же, но при этом проходит: программа не запускалась, оба файла оказывались пустыми, и сравнение пустого с пустым считалось успехом. Добавил условие, что событий должно быть больше десяти.

12 Выводы

Я написал консольную программу, которая моделирует работу группы лифтов в многопоточном исполнении. Каждая сущность с поведением — отдельный поток; при обычных параметрах их получается около тридцати. Потоки работают с общим состоянием под одним мьютексом и будят друг друга семафорами. Дополнительно сделана вторая версия, где семафоры заменены условными переменными: обе собираются из одного набора исходников и ведут себя одинаково.

Все шесть инвариантов проверяются во время работы, завершиться программа может четырьмя способами, включая смену режима по сигналу SIGUSR1. Что всё это действительно работает, подтверждают 38 автоматических проверок, 60 прогонов со случайными конфигурациями и чистый результат детектора гонок на обеих сборках.